

Jaipur Engineering College & Research Centre, Jaipur
Department of Computer Science & Engineering
Internet of Things Lab Manual (7CS4-21) Session 2020-2021



Internet of Things (7CS4-21)
VII Semester
Session 2020-2021

Jaipur Engineering College and Research Centre
Internet of Things (7CS4-21) Session 2020-2021

Vision of the CSE Department

To become renowned Centre of excellence in computer science and engineering and make competent engineers & professionals with high ethical values prepared for lifelong learning.

Mission of the CSE Department

M1: To impart outcome based education for emerging technologies in the field of computer science and engineering.

M2: To provide opportunities for interaction between academia and industry.

M3: To provide platform for lifelong learning by accepting the change in technologies

M4: To develop aptitude of fulfilling social responsibilities.

Program Educational Objectives (PEO)

- 1 To provide students with the fundamentals of Engineering Sciences with more emphasis in **Computer Science & Engineering** by way of analyzing and exploiting engineering challenges.
- 2 To train students with good scientific and engineering knowledge so as to comprehend, analyze, design, and create novel products and solutions for the real life problems.
- 3 To inculcate professional and ethical attitude, effective communication skills, teamwork skills, multidisciplinary approach, entrepreneurial thinking and an ability to relate engineering issues with social issues.
- 4 To provide students with an academic environment aware of excellence, leadership, written ethical codes and guidelines, and the self motivated life-long learning needed for a successful professional career.
- 5 To prepare students to excel in Industry and Higher education by Educating Students along with High moral values and Knowledge

Jaipur Engineering College and Research Centre
Internet of Things (7CS4-21) Session 2020-2021

Program Outcomes

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Jaipur Engineering College and Research Centre
Internet of Things (7CS4-21) Session 2020-2021

Course Outcome: Internet of Things (7CS4-21)

STUDENTS WILL BE ABLE TO:

CO1: Installed and practical expertise using Raspberry Pi or Arduino with Linux commands and python programming.

CO2: Assemble IOT projects using the most popular Raspberry Pi board or Arduino.

Subject	Code	L / T / P	CO	P O 1	P O 2	P O 3	P O 4	P O 5	P O 6	P O 7	P O 8	P O 9	P O 10	P O 11	P O 12	P S O 1	P S O 2
IOT	7CS4-21	P	Installed and practical expertise using Raspberry Pi or Arduino with Linux commands and python programming.	H	H	H	M	H	-	-	L	M	M	M	H	H	H
		P	Assemble IOT projects using the most popular Raspberry Pi board or Arduino.	H	H	H	H	H	L	-	L	L	M	M	H	H	H

Jaipur Engineering College and Research Centre
Internet of Things (7CS4-21) Session 2020-2021

RTU Syllabus

7CS4-21: Internet of Things Lab

Credit: 2
OL+OT+4P

Max. Marks: 100(IA:60, ETE:40)
End Term Exam: 2 Hours

SN	List of Experiments
1	Start Raspberry Pi and try various Linux commands in command terminal window: <i>ls, cd, touch, mv, rm, man, mkdir, rmdir, tar, gzip, cat, more, less, ps, sudo, cron, chown, chgrp, ping etc.</i>
2	Run some python programs on Pi like: a) Read your name and print Hello message with name b) Read two numbers and print their sum, difference, product and division. c) Word and character count of a given string. d) Area of a given shape (rectangle, triangle and circle) reading shape and appropriate values from standard input.
3	Run some python programs on Pi like: a) Print a name 'n' times, where name and n are read from standard input, using for and while loops. b) Handle Divided by Zero Exception. c) Print current time for 10 times with an interval of 10 seconds. d) Read a file line by line and print the word count of each line.
4	a) Light an LED through Python program b) Get input from two switches and switch on corresponding LEDs c) Flash an LED at a given on time and off time cycle, where the two times are taken from a file.
5	a) Flash an LED based on cron output (acts as an alarm) b) Switch on a relay at a given time using cron, where the relay's contact terminals are connected to a load. c) Get the status of a bulb at a remote place (on the LAN) through web.
	The student should have hands on experience in using various sensors like temperature, humidity, smoke, light, etc. and should be able to use control web camera, network, and relays connected to the Pi.

BOOKS:-

1. Text Books:

- Analytics of IOT by Andrew Minter,
- Programming with raspberry Pi, 2nd Edition by Simon Monk
- Programming Arduino, 2nd Edition by Simon Monk

2. Reference Books:

- Arduino Advanced Methods By Ethan Thorpe 2nd Edition

INSTRUCTIONAL METHODS:-

1. Direct Instructions:

- I. Power Point presentation and Videos

2. Interactive Instruction:

- I. Algorithms and their Discussion

3 .Indirect Instructions:

- I. Problem solving by implementing Project ideas

4. ASSESSMENT OF OUTCOMES:-

1. End term Practical exam (Conducted by RTU, KOTA)
2. Daily Lab interaction.

5. OUTCOMES WILL BE ACHIEVED THROUGH FOLLOWING:-

1. Lab Teaching (through Google meet).
2. Discussion on Algorithms.

Jaipur Engineering College and Research Centre
Internet of Things (7CS4-21) Session 2020-2021

Lab Index

S.No	List of Experiment
1	Study and Install Python in Eclipse and WAP for data types in Python
2	Write a Program for arithmetic operation in Python
3	Write a Program for looping statement in Python
4	Study and Install IDE of Arduino and different types of Arduino
5	Write program using Arduino IDE for Blink LED
6	Write Program for RGB LED using Arduino
7	Study the Temperature sensor and Write Program for monitor temperature using Arduino
8	Study and Implement RFID, NFC using Arduino
9	Study and implement MQTT protocol using Arduino
10	Study and Configure Raspberry Pi
11	Write a Program for LED blink using Raspberry Pi
12	Study and Implement Zigbee Protocol using Arduino / Raspberry Pi

INSTRUCTIONS OF LAB

DO's

1. Please switch off the Mobile/Cell phone before entering Lab.
2. Enter the Lab with complete source code and data.
3. Check whether all peripheral are available at your desktop before proceeding for program.
4. Intimate the lab In charge whenever you are incompatible in using the system or in case software get corrupted/ infected by virus.
5. Arrange all the peripheral and seats before leaving the lab.
6. Properly shutdown the system before leaving the lab.
7. Keep the bag outside in the racks.
8. Enter the lab on time and leave at proper time.
9. Maintain the decorum of the lab.
10. Utilize lab hours in the corresponding experiment.
11. Get your Cd / Pen drive checked by lab In charge before using it in the lab.

DON'TS

1. No one is allowed to bring storage devices like Pan Drive /Floppy etc. in the lab.
2. Don't mishandle the system.
3. Don't leave the system on standing for long
4. Don't bring any external material in the lab.
5. Don't make noise in the lab.
6. Don't bring the mobile in the lab. If extremely necessary then keep ringers off.
7. Don't enter in the lab without permission of lab Incharge.
8. Don't litter in the lab.
9. Don't delete or make any modification in system files.
10. Don't carry any lab equipments outside the lab.

We need your full support and cooperation for smooth functioning of the lab.

INSTRUCTIONS FOR STUDENT

BEFORE ENTERING IN THE LAB

- All the students are supposed to prepare the theory regarding the next program.
- Students are supposed to bring the practical file and the lab copy.
- Previous programs should be written in the practical file.
- Any student not following these instructions will be denied entry in the lab.

WHILE WORKING IN THE LAB

- Adhere to experimental schedule as instructed by the lab incharge.
- Get the previously executed program signed by the instructor.
- Get the output of the current program checked by the instructor in the lab copy.
- Each student should work on his/her assigned computer at each turn of the lab.
- Take responsibility of valuable accessories.
- Concentrate on the assigned practical and do not play games.
- If anyone caught red handed carrying any equipment of the lab, then he will have to face serious consequences.

Experiment No 1

Aim: Study and Install Python in Eclipse and WAP for data types in python.

Objectives: Student should get the knowledge of Python and Eclipse background.

Outcomes: Student will be aware of Python and Eclipse background.

Configure Python in Eclipse

Download Python from <http://www.python.org>. Download the version 3.3.1 or higher of Python. If you are using Windows you can use the native installer for Python.

Eclipse Python plug-in

The following assume that you have already Eclipse installed. For an installation description of Eclipse please see Eclipse IDE for Java.

For Python development under Eclipse you can use the PyDev Plugin which is an open source project. Install PyDev via the Eclipse update manager via the following update site: <http://pydev.org/updates>.

Configuration of Eclipse

You also have to maintain in Eclipse the location of your Python installation. Open in the Window ► Preference ► Pydev ► Interpreter Python menu.

Press the New button and enter the path to python.exe in your Python Installation directory. For Linux and Mac OS X users this is normally /usr/bin/python.

Data Types in Python:

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

```
#!/usr/bin/python
```

```
counter = 100      # An integer assignment
```

```
miles = 1000.0     # A floating point
```

```
name = "John"      # A string
```

```
print counter  
print miles  
print name
```

Here, 100, 1000.0 and "John" are the values assigned to *counter*, *miles*, and *name* variables, respectively.

Multiple Assignments

Python allows you to assign a single value to several variables simultaneously.

For example – `a = b = c = 1`

Standard Data Types:

The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Python has five standard data types –

- Numbers
- String
- List
- Tuple
- Dictionary
- Python Numbers

Number data types store numeric values. Number objects are created when you assign a value to them.

For example –

```
var1 = 1
```

```
var2 = 10
```

You can also delete the reference to a number object by using the `del` statement. The syntax of the `del` statement is –

```
del var1[,var2[,var3[....,varN]]]
```

Python supports four different numerical types –

`int` (signed integers)

`long` (long integers, they can also be represented in octal and hexadecimal)

`float` (floating point real values)

`complex` (complex numbers)

Python Strings

Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes. Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.

```
#!/usr/bin/python
```

```
str = 'Hello World!'
```

```
print      str      #   Prints   complete string
```

```
print      str[0]    #   Prints   first character of the string
```

```
print str[2:5]      # Prints characters      starting from 3rd to 5th
```

```
print str[2:]       # Prints string starting from 3rd character
```

```
print  str  * 2     #   Prints   string two   times
```

```
print  str  + "TEST" #Prints   concatenated string
```

This will produce the following result –

Hello World!

H

llo

llo World!

Hello World!Hello World!

Hello World!TEST.

Experiment No 2

Aim: Study and WAP for Arithmetic operation in python.

Objectives: Student should get the knowledge of Arithmetic operation in python.

Outcomes: Student will be developing of Arithmetic operation programs in python.

Assume variable a holds 21 and variable b holds 10, then –
#!/user/bin/python

```
a = 21
```

```
b = 10
```

```
c = 0  
c = a + b
```

```
print "Line 1 - Value of c is ", c
```

```
c = a - b
```

```
print "Line 2 - Value of c is ", c
```

```
c = a * b
```

```
print "Line 3 - Value of c is ", c
```

```
c = a / b
```

```
print "Line 4 - Value of c is ", c
```

```
c = a % b
```

```
print "Line 5 - Value of c is ", c
```

```
a = 2
```

```
b = 3
```

```
c = a**b
```

```
Print "Line 6 - Value of c is ", c
```

a = 10

b = 5

c = a//b

Print "Line 7 - Value of c is ", c

When you execute the above program, it produces the following result –

Line 1 - Value of c is 31

Line 2 - Value of c is 11

Line 3 - Value of c is 210

Line 4 - Value of c is 2

Line 5 - Value of c is 1

Line 6 - Value of c is 8

Line 7 - Value of c is 2

Experiment No 3

Aim: Study and WAP for looping statement in python.

Objectives: Student should get the knowledge of for looping statement in python.

Outcomes: Student will be developing looping statement programs in python.

```
#!/usr/bin/python
```

```
for letter in 'Python':    # First Example
```

```
    print 'Current Letter :', letter
```

```
fruits = ['banana', 'apple', 'mango']
```

```
for fruit in fruits:      # Second Example
```

```
    print 'Current fruit :', fruit
```

```
print "Good bye!"
```

When the above code is executed, it produces the following result –

Current Letter : P

Current Letter : y

Current Letter : t

Current Letter: h

Current Letter : o

Current Letter : n

Current fruit : banana

Current fruit : apple

Current fruit : mango

Good bye!

Iterating by Sequence Index

An alternative way of iterating through each item is by index offset into the sequence itself.

Following is a simple example –

```
#!/usr/bin/python
```

```
fruits = ['banana', 'apple', 'mango']  
for index in range(len(fruits)):  
    print 'Current fruit :', fruits[index]  
print "Good bye!"
```

When the above code is executed, it produces the following result –

Current fruit : banana

Current fruit : apple

Current fruit : mango

Good bye!

Here, we took the assistance of the `len()` built-in function, which provides the total number of elements in the tuple as well as the `range()` built-in function to give us the actual sequence to iterate over.

Using else Statement with Loops

Python supports to have an else statement associated with a loop statement

If the **else** statement is used with a **for** loop, the **else** statement is executed when the loop has exhausted iterating the list.

If the **else** statement is used with a **while** loop, the **else** statement is executed when the condition becomes false.

The following example illustrates the combination of an else statement with a for statement that searches for prime numbers from 10 through 20.

```
#!/usr/bin/pytho

for num in range(10,20):      #to iterate between 10 to 20

    for i in range(2,num):    #to iterate on the factors of the number

        if num%i == 0:       #to determine the first factor

            j=num/i           #to calculate the second factor

            Print '%d equals %d * %d' % (num,i,j)

            Break #to move to the next number, the #first FOR else: #

    else part of the loop

        Print num, 'is a prime number'
```

When the above code is executed, it produces the following result – 10

equals 2 * 5

11 is a prime number

12 equal 2 * 6

13 is a prime number

14 equal 2 * 7

15 equal 3 * 5

16 equal 2 * 8

17 is a prime number

Experiment No 4

Aim: Study and Install IDE of Arduino and different types of Arduino

Objectives: Student should get the knowledge of Arduino IDE and different types of Arduino Board

Outcomes: Student will be get knowledge of Arduino IDE and different types of Arduino Board

Arduino:

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software. It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a ready-made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board. Arduino provides a standard form factor that breaks the functions of the micro-controller into a more accessible package.

The key features are –

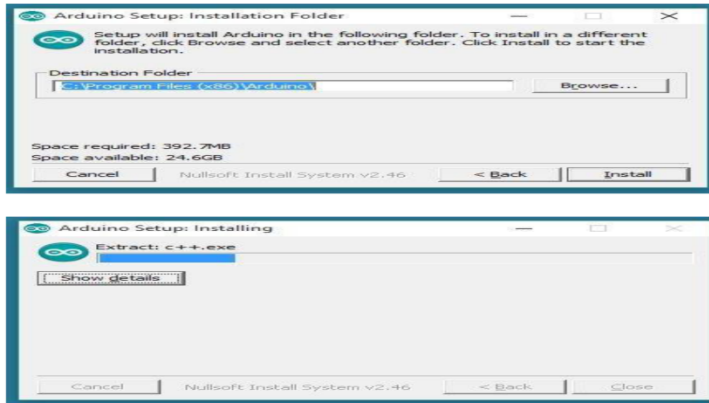
Arduino boards are able to read analog or digital input signals from different sensors and turn it into an output such as activating a motor, turning LED on/off, connect to the cloud and many other actions.

You can control your board functions by sending a set of instructions to the microcontroller on the board via Arduino IDE (referred to as uploading software).

Download the Arduino Software (IDE)

Get the latest version from the arduino.cc web site. You can choose between the Installer (.exe) and the Zip packages. We suggest you use the first one that installs directly everything you need to use the Arduino Software (IDE), including the drivers. With the Zip package you need to install the drivers manually. The Zip file is also useful if you want to create a portable installation.

Choose the components to install



Choose the installation directory (we suggest to keep the default one)

The process will extract and install all the required files to execute properly the Arduino Software (IDE)

Proceed with board specific instructions

Different Arduino Boards:

1.Arduino uno

This is the latest revision of the basic Arduino USB board. It connects to the computer with a standard USB cable and contains everything else you need to program and use the board.

2. Arduino NG REV-C

Revision C of the Arduino NG does not have a built-in LED on pin 13 - instead you'll see two small unused solder pads near the labels "GND" and "13".

Arduino Bluetooth

The Arduino BT is a microcontroller board originally was based on the ATmega168, but now is supplied with the 328, and the Bluegiga WT11 bluetooth module. It supports wireless serial communication over bluetooth.

Arduino Mega

The original Arduino Mega has an ATmega1280 and an FTDI USB-to-serial chip.

Arduino NANO

The Arduino Nano 3.0 has an ATmega328 and a two-layer PCB. The power LED moved to the top of the board.

Experiment No 5

Aim: Write program using Arduino IDE for Blink LED

Objectives: Student should get the knowledge of Arduino Board and different types of LED

Outcomes: Student will be Write program using Arduino IDE for Blink LED

Hardware Requirements:

1x Breadboard

1x Arduino Uno R3 1x

RGB LED

1x 330Ω Resistor

2x Jumper Wires

Blinking the RGB LED

```
1.  /*
2.   Blink
3.   Turns on an LED on for one second, then off for one second, repeatedly.
4.
5.   This example code is in the public domain.
6.   */
7.
8.  // Pin 13 has an LED connected on most Arduino boards.
9.  // give it a name:
10. int led = 13;
11.
12. // the setup routine runs once when you press reset:
13. void setup() {
14.   // initialize the digital pin as an output.
15.   pinMode(led, OUTPUT);
16. }
17.
```

```

18. // the loop routine runs over and over again forever:
19. void loop() {
20.   digitalWrite(led, HIGH);      // turn the LED on (HIGH is the voltage level)
21.   delay(1000);                  // wait for a second
22.   digitalWrite(led, LOW);       // turn the LED off by making the voltage LOW
23.   delay(1000);                  // wait for a second
24. }

```

Let's try using a different pin of the Arduino – say D7. Move the red jumper lead from pin D13 to pin D7 and modify the following line near the top of the sketch:

```
1. int led = 13;
```

so that it reads:

```
1. int led = 7;
```

Upload the modified sketch to your Arduino board and the LED should still be blinking, but this time using pin D7.

Experiment No 6

Aim: Write Program for RGB LED using Arduino.

Objectives: Student should get the knowledge of Arduino IDE and RGB Led

Outcomes: Student will be developed programs using Arduino IDE and Arduino Board for RGB Led

Hardware Requirements

1x Breadboard

1x Arduino Uno

R3 1x LED

1x 330Ω Resistor

2x Jumper

Wires Blinking the

LED

The following test sketch will cycle through the colors red, green, blue, yellow, purple, and aqua. These colors being some of the standard Internet colors.

```
1.  /*
2.  Adafruit Arduino - Lesson 3. RGB LED
3.  */
4.
5.  int redPin = 11;
6.  int greenPin = 10;
7.  int bluePin = 9;
8.
9.  //uncomment this line if using a Common Anode LED
10. //define COMMON_ANODE
11.
12. void setup()
13. {
14.   pinMode(redPin, OUTPUT);
15.   pinMode(greenPin, OUTPUT);
16.   pinMode(bluePin, OUTPUT);
17. }
```

```

18.

19. void loop()
20. {

21.   setColor(255, 0, 0); // red
22.   delay(1000);

23.   setColor(0, 255, 0); // green

24.   delay(1000);
25.   setColor(0, 0, 255); // blue

26.   delay(1000);
27.   setColor(255, 255, 0); // yellow

28.   delay(1000);
29.   setColor(80, 0, 80); // purple

30.   delay(1000);

31.   setColor(0, 255, 255); // aqua
32.   delay(1000);

33. }
34.

35. void setColor(int red, int green, int blue)
36. {

37.   #ifdef COMMON_ANODE

38.     red = 255 - red;
39.     green = 255 - green;

40.     blue = 255 - blue;
41.   #endif

42.   analogWrite(redPin, red);
43.   analogWrite(greenPin, green);

44.   analogWrite(bluePin, blue);

45. }

```

The sketch starts by specifying which pins are going to be used for each of the colors:

[Download file](#)

[Copy Code](#)

1. `int redPin = 11;`
2. `int greenPin = 10;`
3. `int bluePin = 9;`

The next step is to write the 'setup' function. As we have learnt in earlier lessons, the setup function runs just once after the Arduino has reset. In this case, all it has to do is define the three pins we are using as being outputs.

1. `void setup()`
2. `{`
3. `pinMode(redPin, OUTPUT);`
4. `pinMode(greenPin, OUTPUT);`
5. `pinMode(bluePin, OUTPUT);`
6. `}`

Before we take a look at the 'loop' function, lets look at the last function in the sketch.

[Download file](#)

[Copy Code](#)

1. `void setColor(int red, int green, int blue)`
2. `{`
3. `analogWrite(redPin, red);`
4. `analogWrite(greenPin, green);`
5. `analogWrite(bluePin, blue);`
6. `}`

This function takes three arguments, one for the brightness of the red, green and blue LEDs. In each case the number will be in the range 0 to 255, where 0 means off and 255 means maximum brightness. The function then calls 'analogWrite' to set the brightness of each LED.

If you look at the 'loop' function you can see that we are setting the amount of red, green and blue light that we want to display and then pausing for a second before moving on to the next color.

```
1. void loop()
2. {
3.     setColor(255, 0, 0); // red
4.     delay(1000);
5.     setColor(0, 255, 0); // green
6.     delay(1000);
7.     setColor(0, 0, 255); // blue
8.     delay(1000);
9.     setColor(255, 255, 0); // yellow
10.    delay(1000);
11.    setColor(80, 0, 80); // purple
12.    delay(1000);
13.    setColor(0, 255, 255); // aqua
14.    delay(1000);
15. }
```

Experiment No 7

Aim: Study the Temperature sensor and Write Program for monitor temperature using Arduino.

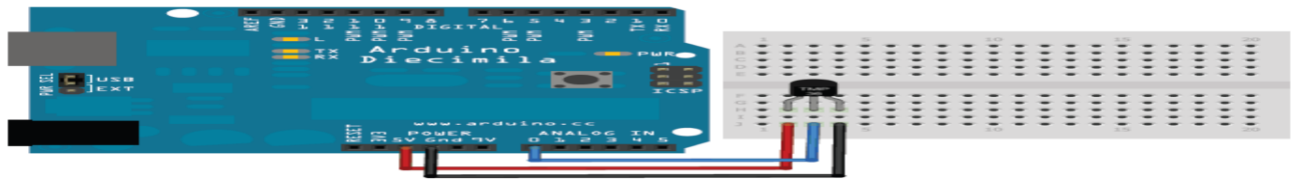
Objectives: Student should get the knowledge of Temperature Sensor

Outcomes: Student will be developed programs using Arduino IDE and Arduino Board for Temperature Sensor

Connecting to a Temperature Sensor

These sensors have little chips in them and while they're not that delicate, they do need to be handled properly. Be careful of static electricity when handling them and make sure the power supply is connected up correctly and is between 2.7 and 5.5V DC - so don't try to use a 9V battery!

They come in a "TO-92" package which means the chip is housed in a plastic hemi-cylinder with three legs. The legs can be bent easily to allow the sensor to be plugged into a breadboard. You can also solder to the pins to connect long wires. If you need to waterproof the sensor, you can see below for an Instructable for how to make an excellent case.



If you're using a 5V Arduino, and connecting the sensor directly into an Analog pin, you can use these formulas to turn the 10-bit analog reading into a temperature:

$$\text{Voltage at pin in milliVolts} = (\text{reading from ADC}) * (5000/1024)$$

This formula converts the number 0-1023 from the ADC into 0-5000mV (= 5V)

If you're using a 3.3V Arduino, you'll want to use this:

$$\text{Voltage at pin in milliVolts} = (\text{reading from ADC}) * (3300/1024)$$

This formula converts the number 0-1023 from the ADC into 0-3300mV (= 3.3V)

Then, to convert mill volts into temperature, use this formula:

$$\text{Centigrade temperature} = [(\text{analog voltage in mV}) - 500] / 10$$

Simple Thermometer

This example code for Arduino shows a quick way to create a temperature sensor, it simply prints to the serial port what the current temperature is in both Celsius and Fahrenheit.

```
1. //TMP36 Pin Variables
2. int sensorPin = 0; //the analog pin the TMP36's Vout (sense) pin is connected
3.                               //the resolution is 10 mV / degree centigrade with a
4.                               //500 mV offset to allow for negative temperatures
5.
6. /*
7.  * setup() - this function runs once when you turn your Arduino on
8.  * We initialize the serial connection with the computer
9.  */
10. void setup()
11. {
12.   Serial.begin(9600); //Start the serial connection with the computer
13.                       //to view the result opens the serial monitor
14. }
15.
16. void loop()              // run over and over again
17. {
18.   //getting the voltage reading from the temperature sensor
19.   int reading = analogRead(sensorPin);
20.
21.   // converting that reading to voltage, for 3.3v arduino use 3.3
22.   float voltage = reading * 5.0;
23.   voltage /= 1024.0;
24.
25.   // print out the voltage
26.   Serial.print(voltage); Serial.println(" volts");
27.
28.   // now print out the temperature
29.   float temperatureC = (voltage - 0.5) * 100 ; //converting from 10 mv per degree
```

with 500 mV offset

30. //to degrees ((voltage - 500mV)

Times 100)

31. Serial.print(temperatureC); Serial.println(" degrees C");

32.

33. // now convert to Fahrenheit

34. float temperatureF = (temperatureC * 9.0 / 5.0) + 32.0;

35. Serial.print(temperatureF); Serial.println(" degrees F");

36.

37. delay(1000);

38. }

Experiment No 8

Aim: Study and Implement RFID, NFC using Arduino.

Objectives: Student should get the knowledge of RFID, NFC using Arduino.

Outcomes: Student will be developed programs using Arduino IDE and Arduino Board for RFID, NFC.

Hardware Requirements:

- 1 x Arduino UNO or 1 x Starter Kit for Raspberry Pi + Raspberry Pi
- 1 x Communication Shield

- 1 x RFID 13.56 MHz / NFC Module for Arduino and Raspberry Pi
- 1 x Mifare tag (card/keyring/sticker)
- 1 x PC

RFID:

RFID system is made up of two parts: a tag or label and a reader. RFID tags or labels are embedded with a transmitter and a receiver. The RFID component on the tags have two parts: a microchip that stores and processes information, and an antenna to receive and transmit a signal. The tag contains the specific serial number for one specific object.

How to Interface RFID Reader to Arduino

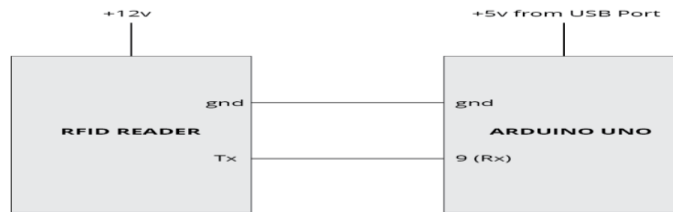
Take note of the following stuffs.

Note 1:- Power supply requirement of RFID Readers vary from product to product. The RFID reader I used in this tutorial is a 12 Volts one. There are 5 Volts and 9 Volts versions available in the market.

Note 2:- You may ensure the RFID Reader and RFID Tags are frequency compatible. Generally they are supposed to be 125 KHz. You may ensure this before purchasing them.

Note 3:- There are two possible outputs from an RFID Reader. One is RS232 compatible output and other one is TTL compatible output. A TTL compatible output pin can be connected directly to Arduino. Whereas an RS232 compatible output must be converted

So that's all! Lets get to circuit diagram!



```
#include <SoftwareSerial.h>

SoftwareSerial mySerial(9, 10);

void setup()
{
  mySerial.begin(9600); // Setting the baud rate of Software Serial Library
  Serial.begin(9600); //Setting the baud rate of Serial Monitor
}

void loop()
{
  if(mySerial.available()>0)
  {
```



```
Serial.write(mySerial.read());  
}  
}
```

mySerial.available() – checks for any data coming from RFID reader module through the SoftwareSerial pin 9. Returns the number of bytes available to read from software serial port. Returns a -1 if no data is available to read.

mySerial.read() – Reads the incoming data through software serial port.

Serial.write() – Prints data to serial monitor of arduino. So the function Serial. Write (mySerial.read ()) – prints the data collected from software serial port to serial monitor of arduino.

Experiment No 9

Aim: Study and Implement MQTT Protocol using Arduino.

Objectives: Student should get the knowledge of MQTT Protocol using Arduino.

Outcomes: Student will be developed programs using Arduino IDE and Arduino Board for MQTT Protocol

MQTT:

MQ Telemetry Transport (MQTT) is an open source protocol for constrained devices and low-bandwidth, high-latency networks. It is a publish/subscribe messaging transport that is extremely lightweight and ideal for connecting small devices to constrained networks.

MQTT is bandwidth efficient, data agnostic, and has continuous session awareness. It helps minimize the resource requirements for your IoT device, while also attempting to ensure reliability and some degree of assurance of delivery with grades of service.

MQTT methods: MQTT defines methods (sometimes referred to as *verbs*) to indicate the desired action to be performed on the identified resource.

Connect

Waits for a connection to be established with the server.

Disconnect

Waits for the MQTT client to finish any work it must do, and for the TCP/IP session to disconnect.

Subscribe

Waits for completion of the Subscribe or UnSubscribe method.

Unsubscribe

Requests the server unsubscribe the client from one or more topics.

Publish

Returns immediately to the application thread after passing the request to the MQTT client.

Experiment No 10

Aim: Study and Configure Raspberry Pi.

Objectives: Student should get the knowledge of Raspberry Pi.

Outcomes: Student will be get knowledge of Raspberry Pi

Raspberry Pi

The **Raspberry Pi** is a series of small single-board computers developed in the United Kingdom by the Foundation to promote the teaching of basic computer science in schools and in developing countries. The original model became far more popular than anticipated, selling outside of its target market for uses such as robotics. Peripherals (including keyboards, mice and cases) are not included with the Raspberry Pi. Some accessories however have been included in several official and unofficial bundles.

How to get and install NOOBS

DOWNLOAD NOOBS OS FROM

We recommend using an SD card with a minimum capacity of 8GB.

1. GO to the <https://www.raspberrypi.org/downloads/>
2. Click on NOOBS, then click on the Download ZIP button under 'NOOBS (offline and network install)', and select a folder to save it to.
3. Extract the files from the zip.

FORMAT YOUR SD CARD

It is best to format your SD card before copying the NOOBS files onto it. To do this:

1. Download SD Formatter 4.0 for either Windows or Mac.
2. Follow the instructions to install the software.
3. Insert your SD card into the computer or laptop's SD card reader and make a note of the drive letter allocated to it, e.g. G:/
4. In SD Formatter, select the drive letter for your SD card and format it.

DRAG AND DROP NOOBS FILES

1. Once your SD card has been formatted, drag all the files in the extracted NOOBS folder and drop them onto the SD card drive.
2. The necessary files will then be transferred to your SD card.
3. When this process has finished, safely remove the SD card and insert it into your Raspberry Pi.

FIRST BOOT

1. Plug in your keyboard, mouse, and monitor cables.
2. Now plug the USB power cable into your Pi.
3. Your Raspberry Pi will boot, and a window will appear with a list of different operating systems that you can install. We recommend that you use Raspbian – tick the box next to Raspbian and click on Install.
4. Raspbian will then run through its installation process. Note that this can take a while.
5. When the install process has completed, the Raspberry Pi configuration menu (raspi-config) will load. Here you are able to set the time and date for your region, enable a Raspberry Pi camera board, or even create users. You can exit this menu by using **Tab** on your keyboard to move to Finish.

LOGGING IN AND ACCESSING THE GRAPHICAL USER INTERFACE

The default login for Raspbian is username pi with the password raspberry. Note that you will not see any writing appear when you type the password. This is a security feature in Linux.

To load the graphical user interface, type startx and press Enter.

Experiment No 11

Aim: WAP for LED blink using Raspberry Pi.

Objectives: Student should get the knowledge of LED blinking using Raspberry Pi.

Outcomes: Student will be developed program of LED bilking using Raspberry Pi.

Hardware Requirements:

1x Breadboard 1x

Raspberry Pi 1x

RGB LED

1x 330 Ω Resistor

2x Jumper Wires

In this experiment, connect a 220 Ω resistor to the anode of the LED, then the resistor to 3.3 V and connect the cathode of the LED to GPIO0 (See Raspberry Pi Pin Number Introduction). Write 1 to GPIO0, and the LED will stay off; write 0 to GPIO0, and then the LED will blink, just as indicated by the principle



Step 1: Build the circuit given above

Step 2: Change directory

cd /home/pi/Sunfounder_SuperKit_Python_code_for_RaspberryPi/

Step 3: Run

sudo python 01_led.py

Now, you should see the LED blink.

Python Code

```
#!/usr/bin/env python
```

```
import RPi.GPIO as GPIO
```

```
import time
```

```
LedPin = # pin11  
11
```

```
def setup():
```

```
    GPIO.setmode(GPIO.BOARD) # Numbers GPIOs by physical location
```

```
    GPIO.setup(LedPin, GPIO.OUT) # Set LedPin's mode is output
```

```
    # Set LedPin high(+3.3V) to off led
```

```
    GPIO.output(LedPin,  
GPIO.HIGH) #
```

```
def loop():
```

```
    while True:
```

```
        print '...led on'
```

```
GPIO.output(LedPin, GPIO.LOW)    # led on
```

```
time.sleep(0.5)
```

```
print 'led off...'
```

```
GPIO.output(LedPin, GPIO.HIGH) # led off
```

```
time.sleep(0.5)
```

```
def destroy():
```

```
    GPIO.output(LedPin,  
    GPIO.HIGH)                # led off
```

```
    GPIO.cleanup()            # Release resource
```

```
if __name__ == '__main__':    # Program start from here
```

```
    Setup ()
```

```
    Try:
```

```
    loop()
```

Except Keyboard Interrupt: # When 'Ctrl+C' is pressed, the child program destroy () will be executed.

```
    Destroy ()
```

Experiment No 12

Aim: Study and Implement Zigbee Protocol using Raspberry Pi.

Objectives: Student should get the knowledge of Zigbee Protocol using Raspberry Pi.

Outcomes: Student will be developed program of Zigbee Protocol using Raspberry Pi.

Hardware Requirements

Raspberry Pi2

XBee 1mW Wire Antenna- Series 1 (2 No :)

XBee Explorer Dongle (2 No :)

ZigBee Communication Using Raspberry Pi:

ZigBee is a communication device used for the data transfer between the controllers, computers, systems, really anything with a serial port. As it works with low power consumption, the transmission distances is limited to 10–100 meters line-of-sight. ZigBee devices can transmit data over long distances by passing data through a mesh network of intermediate devices to reach more distant ones. ZigBee is typically used in low data rate applications that require long battery life and secure networking. Its main applications are in the field of wireless sensor network based on industries as it requires short-range low-rate wireless data transfer. The technology defined by the ZigBee specification is intended to be simpler and less expensive than other wireless networks.

 JAIPUR ENGINEERING COLLEGE AND RESEARCH CENTRE	Jaipur Engineering College and Research Centre, Shri Ram ki Nangal, via Sitapura RIICO Jaipur- 302022	Academic year 2019-20
--	---	------------------------------

Python script to perform Zigbee communication which is given below.

```
import serial
# Enable USB Communication
Ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=.5)
While True:
    ser.write('Hello User \r\n')      # write a Data
    Incoming = ser.readline().strip()
    Print 'Received Data : '+ incoming
```

The two zigbee must be in a line of sight and check the results in the Python shell and in the HyperTerminal of the computer.